

1. 题目深度解构

题意重述

去掉背景后，问题是：

有两类事件，按时间顺序发生：

- 1. **W**：新增一个工人。此时可以把所有工人重新任意分配到已有工厂，或者让某些工人闲置。
- 2. **F x**：新增一个产能为 **x** 的工厂。此时只能把“当前闲置的工人”分配到这个新工厂，不能调整已经在工作的工人。

每天事件结束后，所有有工人工作的工厂产生收益，收益等于该工厂产能。

目标是在已知未来所有事件的情况下，最大化总收益。

约束分析

单测：

$$n \leq 2 \times 10^5$$

所有测试总和：

$$\sum n \leq 10^6$$

所以需要接近：

$$O(n \log n)$$

不能做每日暴力重分配，也不能维护所有可能状态。

这题的特殊限制是：

- 只有 **W** 事件允许全局重新分配。
- **F** 事件只能使用闲置工人，不能重新分配已有工人。
- 所有工厂一旦出现，会永久存在。

因此，**W** 事件天然就是“分段点”。

题目类型判定

这是一道偏思维的贪心 + 数据结构题。

核心是：

- 按 **W** 事件分段；
 - 每段内部转化为“选若干个最大权值物品”；
 - 用树状数组维护历史工厂的前若干大产能和；
 - 用二分优化当前段中新工厂与历史工厂的选择数量。
-

2. 思维路径推导

从朴素想法开始

最直接的想法是模拟每天的最优分配。

如果每天都能随便重分配，那么答案很简单：每天选择已有工厂中产能最大的前 `工人数` 个。

但题目麻烦在于：

- `W` 日可以全局重排；
- `F` 日不能重排，只能把闲置工人放进新工厂。

例如样例一中：

```
F 1
F 2
W
F 100
```

第 3 天来了一个工人。当前已有工厂 `1, 2`。

如果只看当天，应该让工人去工厂 `2`，当天收益为 `2`。

但最优方案是第 3 天让工人闲置，第 4 天让他去工厂 `100`，收益更大。

所以，不能做“每天选当前最大”的贪心。

关键观察：`W` 是重置点

因为每次 `W` 事件后，你都可以重新安排所有工人，所以在两个相邻 `W` 之间，之前的具体分配不会影响下一个区间。

设某个 `W` 出现在第 `s` 天，下一个 `W` 前一天是第 `e` 天。

这个区间是：

$$[s, e]$$

设当前已经有 `k` 个工人，区间长度为：

$$L = e - s + 1$$

在这个区间内：

- 第 `s` 天可以把工人分配到所有旧工厂；
- 之后如果来了新工厂，只能用闲置工人去分配；
- 在下一个 `W` 之前，已经工作的工人不能换地方。

因此，在一个区间内，一个工人本质上只能选择一个“工作目标”：

1. 选择某个旧工厂，从第 `s` 天一直工作到第 `e` 天，收益为：

$$x \cdot L$$

1. 选择区间中新出现的某个工厂，假设它在第 d 天出现，则从第 d 天工作到第 e 天，收益为：

$$x \cdot (e - d + 1)$$

所以，一个区间就变成了：

有若干个候选物品，每个物品有一个收益，最多选 k 个，求最大收益和。

这就是本题的 Aha moment。

区间内的物品价值

对于区间 $[s, e]$ ：

旧工厂：

$$value = x \cdot L$$

区间中新工厂：

$$value = x \cdot (e - d + 1)$$

直接把所有候选值放一起取前 k 大即可。

但旧工厂数量可能很多，每个区间都重新排序会超时。

如何高效处理旧工厂？

对于当前区间，所有旧工厂的价值都有共同因子 L ：

$$x_1 L, x_2 L, x_3 L, \dots$$

所以旧工厂之间的相对大小只由 x 决定。

我们只需要维护所有旧工厂产能 x 的多重集合，并支持：

- 查询前 t 大产能之和；
- 查询第 t 大产能。

用坐标压缩 + 树状数组即可。

如何把当前区间的新工厂合并进去？

设当前区间中新工厂的收益值排序后为：

$$b_1 \geq b_2 \geq \dots \geq b_m$$

如果从这些新工厂里选 q 个，那么一定选前 q 大。

剩下要从旧工厂里选：

$$need - q$$

个。

其中：

$$need = \min(k, \text{旧工厂数量} + \text{新区间工厂数量})$$

于是目标函数是：

$$f(q) = \sum_{i=1}^q b_i + L \cdot \text{TopOldSum}(need - q)$$

需要在合法范围内最大化 $f(q)$ 。

合法范围：

$$\max(0, need - oldCnt) \leq q \leq \min(need, m)$$

因为如果旧工厂不够，就必须多选一些新工厂。

为什么可以二分 q ？

看增量：

$$f(q) - f(q - 1)$$

从选 $q-1$ 个新工厂变成选 q 个新工厂，本质是：

- 加入第 q 大的新工厂收益 b_q ；
- 少选一个旧工厂。

被替换掉的是旧工厂中第：

$$need - q + 1$$

大的旧工厂。

所以：

$$f(q) - f(q - 1) = b_q - L \cdot old_{need-q+1}$$

其中 old_i 表示旧工厂中第 i 大的产能。

随着 q 增大：

- b_q 不增；
- $old_{need-q+1}$ 不减。

所以增量单调不增。

因此 $f(q)$ 是离散凸峰/单峰函数，可以二分找到最后一个正增量的位置。

3. Trick 与 陷阱

Trick 点拨

核心 Trick 是：

把两个 w 之间的一整段，看成若干个“工人槽位”去选择物品。

一个工人槽位在当前段中只能被使用一次：

- 要么一开始就去旧工厂；
- 要么保持闲置，等某个新工厂出现后去那里。

因此区间内部不是动态规划，而是直接取最大收益物品。

第二个 Trick 是：

旧工厂在同一区间里统一乘以区间长度 L ，所以旧工厂的相对顺序永远按产能 x 排序。

这让我们可以只维护历史工厂产能，而不是维护每个区间的实际收益。

易错点

1. F 在第 d 天出现后，当天就可以生产，所以收益天数是：

$$e - d + 1$$

不是 $e - d$ 。

1. 第一个 w 之前出现的工厂没有收益，但它们是第一个 w 区间的旧工厂，不能丢掉。
2. 连续两个 w 之间的区间长度是 1 ，因为前一个 w 当天也要生产。
3. 答案必须用 `long long`。
4. 当前区间结束后，区间中出现的新工厂才加入“旧工厂集合”，供后续区间使用。

4. 代码实现 C++23

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Fenwick {
    int n;
    vector<i64> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, i64 v) {
        for (int i = x; i <= n; i += i & -i) {
            a[i] += v;
        }
    }
}
```

```

}

i64 sum(int x) const {
    i64 r = 0;
    for (int i = x; i > 0; i -= i & -i) {
        r += a[i];
    }
    return r;
}

int kth(i64 k) const {
    int x = 0;
    int p = 1;
    while (p * 2 <= n) {
        p *= 2;
    }
    for (int i = p; i; i >>= 1) {
        if (x + i <= n && a[x + i] < k) {
            x += i;
            k -= a[x];
        }
    }
    return x + 1;
}

};

void solve() {
    int n;
    cin >> n;

    vector<char> op(n);
    vector<int> x(n);
    vector<int> xs;

    for (int i = 0; i < n; i++) {
        cin >> op[i];
        if (op[i] == 'F') {
            cin >> x[i];
            xs.push_back(x[i]);
        }
    }

    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());

    Fenwick cnt(xs.size(), bit(xs.size()));
    i64 oldCnt = 0, oldSum = 0;

    auto add = [&](int v) {
        int p = lower_bound(xs.begin(), xs.end(), v) - xs.begin() + 1;
        cnt.add(p, 1);
        bit.add(p, v);
        oldCnt++;
    };

```

```

        oldSum += v;
    };

    auto sumSmall = [&](i64 k) -> i64 {
        if (k <= 0) {
            return 0;
        }
        if (k >= oldCnt) {
            return oldSum;
        }

        int p = cnt.kth(k);
        i64 c = cnt.sum(p - 1);
        i64 s = bit.sum(p - 1);

        return s + (k - c) * xs[p - 1];
    };

    auto topSum = [&](i64 k) -> i64 {
        if (k <= 0) {
            return 0;
        }
        if (k >= oldCnt) {
            return oldSum;
        }
        return oldSum - sumSmall(oldCnt - k);
    };

    auto kthLarge = [&](i64 k) -> i64 {
        int p = cnt.kth(oldCnt - k + 1);
        return xs[p - 1];
    };

    i64 ans = 0;
    i64 workers = 0;

    int i = 0;

    while (i < n && op[i] == 'F') {
        add(x[i]);
        i++;
    }

    while (i < n) {
        workers++;

        int s = i;
        int j = i + 1;

        while (j < n && op[j] == 'F') {
            j++;
        }
    }

```

```

int e = j - 1;
int L = e - s + 1;

vector<i64> b;
vector<int> addv;

b.reserve(e - s);
addv.reserve(e - s);

for (int p = s + 1; p <= e; p++) {
    int d = e - p + 1;
    b.push_back(1LL * x[p] * d);
    addv.push_back(x[p]);
}

sort(b.begin(), b.end(), greater<>());

int m = b.size();
vector<i64> pref(m + 1);

for (int p = 0; p < m; p++) {
    pref[p + 1] = pref[p] + b[p];
}

i64 need = min<i64>(workers, oldCnt + m);

int lo = max<i64>(0, need - oldCnt);
int hi = min<i64>(need, m);
int take = lo;

auto gain = [&](int q) -> i64 {
    return b[q - 1] - 1LL * L * kthLarge(need - q + 1);
};

int l = lo + 1, r = hi;

while (l <= r) {
    int mid = (l + r) / 2;
    if (gain(mid) > 0) {
        take = mid;
        l = mid + 1;
    } else {
        r = mid - 1;
    }
}

ans += pref[take] + 1LL * L * topSum(need - take);

for (auto v : addv) {
    add(v);
}

i = j;

```



```

    }

    cout << ans << '\n';
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;

    while (T--) {
        solve();
    }

    return 0;
}

```

复杂度：

$$O(n \log n)$$

每个工厂只会进入一次排序所在的区间，总排序复杂度为：

$$\sum m_i \log m_i \leq O(n \log n)$$

树状数组查询和插入也是 $O(\log n)$ 。

5. 总结与启示

难度评分

预估 Codeforces Rating：

$$2300 \sim 2400$$

这题难点不在代码，而在把动态过程切成独立区间，并把区间内策略抽象成“选择最大权值物品”。

核心启示

遇到“某种事件允许全局重置”的题，要优先考虑按这些事件分段；段内状态往往可以被压缩成一次静态选择问题。

同类习题推荐

1. AtCoder ABC306 E - Best Performances
练习动态维护前 k 大元素和。
2. AtCoder ABC281 E - Least Elements
练习滑动窗口中维护前 k 小元素和。
3. Codeforces 865D - Buy Low Sell High
练习把选择过程抽象成“可替换槽位”的贪心模型。

